



Modeling and Simulation of System CS572

Simulation-Based Resource Optimization of a Retail Bank Branch

Author:

Aditya Kant (2203301)

Under the Supervision of

Dr. Neha Karanjkar

**Department of Computer Science and Engineering
Indian Institute of Technology Goa**

April 2026

Contents

1	Project Description: What, Why, and How	2
2	Methodology and Implementation Details	2
2.1	System Architecture & Queue Discipline	2
2.2	The Arrival Process (Non-Homogeneous Poisson Process)	3
2.3	Service Time Distributions	3
2.4	Modeling Customer Psychology (System Friction)	3
2.5	Financial Assumptions & The Objective Function	4
3	Results and Evaluation	4
3.1	Input Validation	4
3.2	System Failure Under Stress (Baseline vs. Optimized)	5
3.3	Human Impact: Customer Abandonment	5
3.4	Financial Evaluation and Optimization	6
4	AI Tool Disclosure	7
4.1	AI parts	7
4.2	Manual Part	7
5	References	8

1 Project Description: What, Why, and How

What: This project is a Discrete Event Simulation (DES) designed to optimize the staffing configuration (Tellers, Helpdesk, and Relationship Managers) of a retail bank branch. It acts as a “digital twin,” meticulously tracking virtual customers from arrival to departure to optimize business and retail flow.

Why: Traditional staffing models rely on steady-state averages (such as the Erlang C formula), which assume constant traffic and infinite customer patience. This leads to severe understaffing during peak operational hours. When a branch is understaffed, high-value retail and business customers experience prolonged wait times and ultimately abandon the queue. The motivation for this project is to demonstrate that minimizing daily operational expenses (OpEx) through reduced staffing is a strategically flawed approach. It incurs substantial opportunity costs via lost revenue from customer walkouts, severely impacting long-term customer relationships.

How: A stochastic simulation was engineered using Python and the `simpy` framework. The system generates realistic traffic waves, injects mathematical models to represent human frustration, and autonomously tests dozens of staffing configurations via a Grid Search to identify the absolute peak of Net Daily Profit.

2 Methodology and Implementation Details

To ensure the simulation accurately mimics a real-world bank environment, advanced probability models were utilized in place of static averages. Figure 1 outlines the high-level flow of the simulation model.

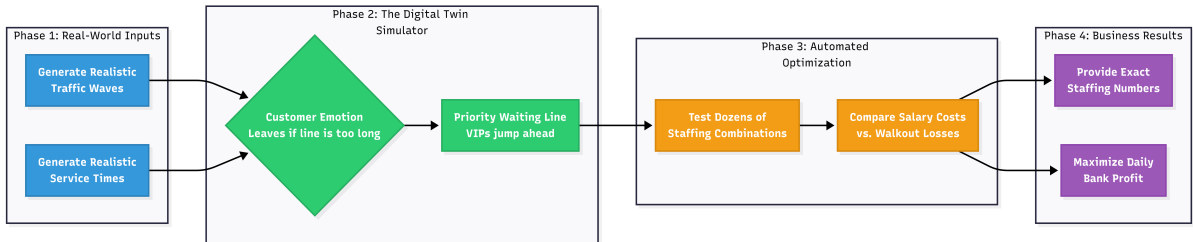


Figure 1: High-Level Architecture of the Simulation Model

2.1 System Architecture & Queue Discipline

The branch is modeled as a multi-server, multi-class queuing network. A **Priority Queue Discipline** is implemented, where 10% of generated customers are randomly designated as high-priority (VIPs). This ensures that high-value relationship clients preempt standard retail traffic.

2.2 The Arrival Process (Non-Homogeneous Poisson Process)

Customer arrivals are non-stationary. A **Non-Homogeneous Poisson Process (NHPP)** generated via the Thinning Algorithm is utilized to simulate realistic traffic waves. The arrival rate at any minute, $\lambda(t)$, is defined as:

$$\lambda(t) = \lambda_{base} \times \delta_{day} \times \tau_{time} \quad (1)$$

- **Base Rate (λ_{base}):** Absolute peak threshold set to 2.5 customers/minute.
- **Day Multiplier (δ_{day}):** Adjusts for weekly seasonality (e.g., Fridays = 1.3 for the end-of-week rush; Mondays = 1.1 for weekend catch-up).
- **Time Multiplier (τ_{time}):** Adjusts for intraday behavior (e.g., 10 AM Opening Rush = 1.2; 12 PM Corporate Lunch Rush = 1.3; 4 PM Taper = 0.6).

2.3 Service Time Distributions

Service times (S_i) are modeled using continuous probability distributions tailored to task complexity:

- **Tellers (60% of traffic):** Modeled with a Gamma distribution ($S_{teller} \sim \Gamma(k = 3.0, \theta = 1.0)$), yielding an expected time of ~ 3 minutes with low variance.
- **Helpdesk (25% of traffic):** Modeled with a Gamma distribution ($S_{hd} \sim \Gamma(k = 5.0, \theta = 1.2)$), yielding an expected time of ~ 6 minutes.
- **Relationship Managers (15% of traffic):** Modeled with a Lognormal distribution ($S_{rm} \sim \text{Lognormal}(\mu = 2.6, \sigma = 0.5)$) to accurately capture the “long tail” of complex business and financial consultations.

2.4 Modeling Customer Psychology (System Friction)

Two distinct “human emotion” failure states are programmed into the customer lifecycle to reflect realistic retail friction:

1. **Logistic Balking (Crowd Avoidance):** Upon arrival, customers evaluate the queue length (Q). Their probability of joining (P_{join}) decays logarithmically relative to a random personal tolerance (K):

$$P_{join} = \frac{1}{1 + e^{0.8(Q-K)}} \quad (2)$$

2. **Weibull Reneging (Patience Expiration):** Once in the queue, a patience limit ($T_{patience}$) is drawn from a Weibull distribution ($\alpha = 2.0$). If a customer’s wait time strictly exceeds this limit before service begins, they abandon the queue, resulting in a lost relationship opportunity.

2.5 Financial Assumptions & The Objective Function

The financial logic relies on comparing daily salaries against the revenue generated per service type:

- **Tellers:** Salary = INR 1,500/day — Value per customer served = INR 150
- **Helpdesk:** Salary = INR 2,000/day — Value per customer served = INR 600
- **Relationship Managers (RM):** Salary = INR 4,000/day — Value per customer served = INR 6,500

The simulation executes a Grid Search to find the staffing configuration (c) that maximizes Net Daily Profit (Z):

$$\text{Maximize } Z(c) = E[R_{served}] - E[C_{opex}] - E[L_{abandoned}] \quad (3)$$

Where R is realized revenue, C is deterministic salary costs, and L is the opportunity cost of customer walkouts.

3 Results and Evaluation

To ensure statistical rigor, the simulation utilized **Common Random Numbers (CRN)** and executed 15 independent replications (equivalent to 4 weeks of operations) to generate 95% Confidence Intervals.

3.1 Input Validation

Figure 2 demonstrates the successful implementation of the NHPP model. The heatmap clearly exhibits realistic banking traffic, accurately capturing the Friday afternoon peaks and the daily corporate lunch rushes.

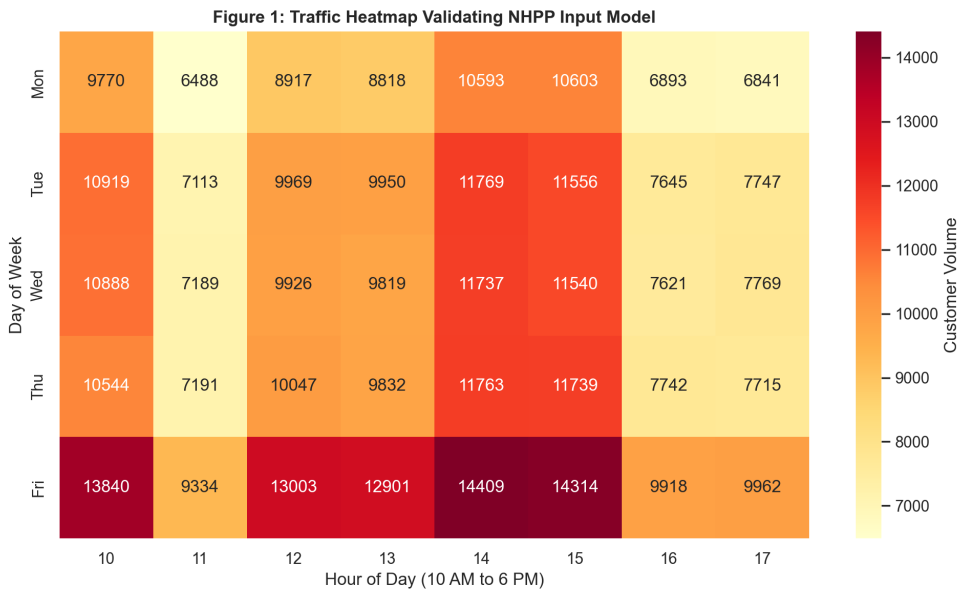


Figure 2: Traffic Heatmap Validating NHPP Input Model

3.2 System Failure Under Stress (Baseline vs. Optimized)

Under the Baseline configuration (3 Tellers, 1 HD, 1 RM), the queue backlog collapses during the midday rush (Figure 3), routinely exceeding 15+ waiting customers. Furthermore, Figure 4 illustrates that baseline wait times frequently violate the 15-minute Service Level Agreement (SLA). The optimized system drastically compresses this variance, ensuring smoother customer flow.

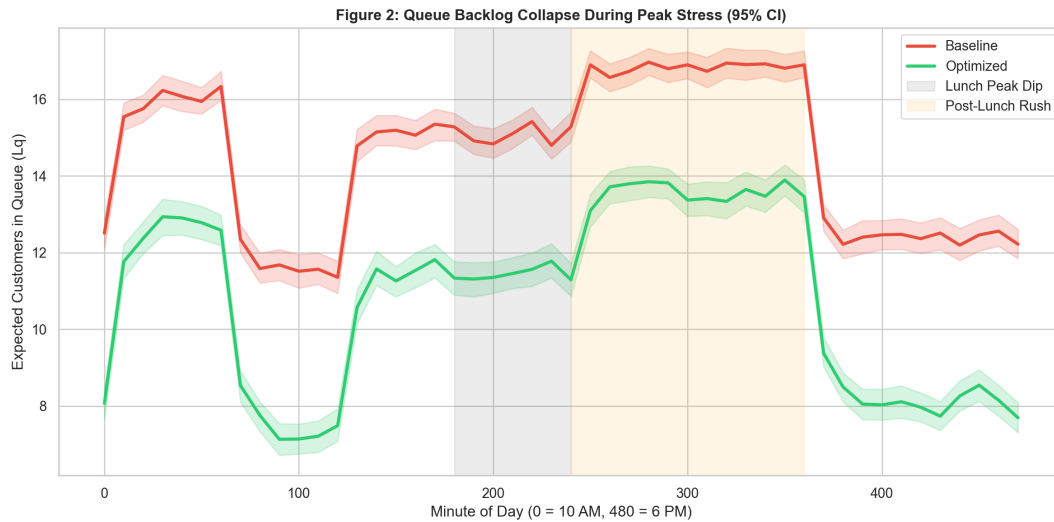


Figure 3: Queue Backlog Collapse During Peak Stress (95% CI)

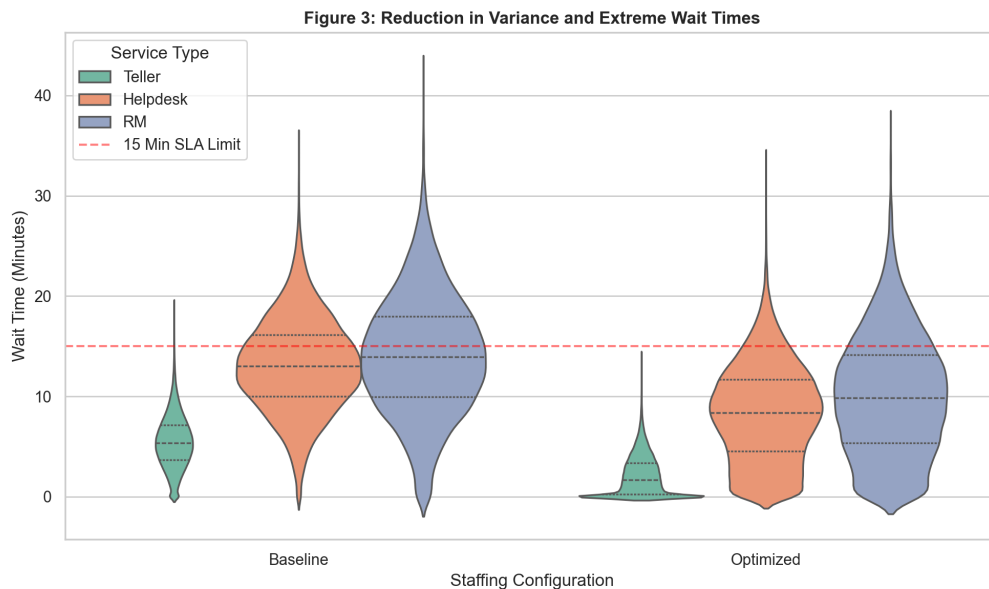


Figure 4: Reduction in Variance and Extreme Wait Times

3.3 Human Impact: Customer Abandonment

The physical consequence of the queue backlog is visualized in Figure 5. The baseline system hemorrhages high-value customers across all service tiers. Optimized staffing

drops walkouts significantly, particularly retaining valuable RM clients.

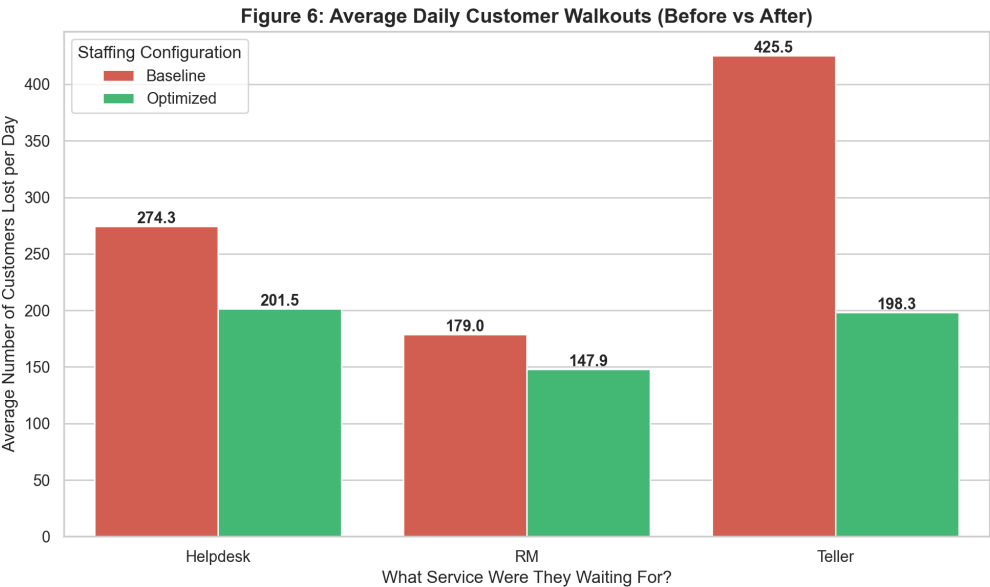


Figure 5: Average Daily Customer Walkouts (Baseline vs. Optimized)

3.4 Financial Evaluation and Optimization

Figure 6 visualizes the core business evaluation. An RM walkout penalizes the bank INR 6,500, while a Teller’s entire daily salary is only INR 1,500. By increasing OpEx marginally, the bank successfully absorbs traffic shocks, drastically reducing walkouts and maximizing net operating profit.

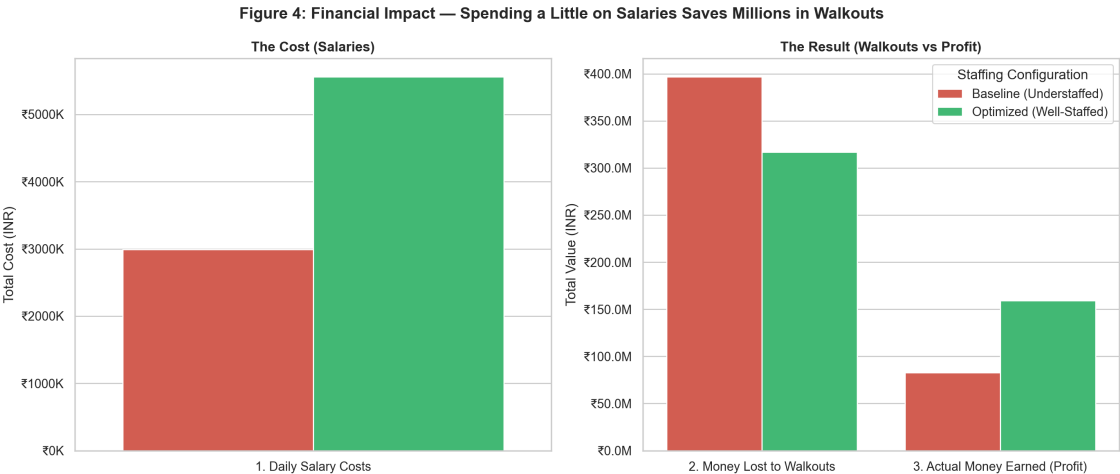


Figure 6: Financial Impact — Spending a Little on Salaries Saves Millions in Walkouts

The Grid Search (Figure 7) evaluated the objective function across a matrix of configurations. The mathematical peak, demonstrating the highest daily profit (INR 612,693), was definitively found at **7 Tellers, 3 Helpdesk, and 2 RMs**.



Figure 7: Grid Search Optimization Landscape

4 AI Tool Disclosure

4.1 AI parts

- **Tool Used:** Gemini (Google)
- **Conceptualization:** AI was used as a tutor to help structure the Operations Research methodology, specifically suggesting the use of Logistic distributions for balking, Weibull distributions for patience reneging, and the framework for the Grid Search marginal analysis.
- **Code Debugging:** AI was utilized to help identify and resolve specific Python errors during the implementation of the `simpy` framework.
- **Report Writing:** AI was utilized strictly as a syntax engine to translate the manual plain-text report into formal LaTeX code.

4.2 Manual Part

- **Project Architecture:** I defined the core business problem (OpEx vs. Opportunity Cost) and designed the "Digital Twin" framework using Discrete Event Simulation rather than static averages.
- **Mathematical Modeling:** I formulated the traffic model using a Non-Homogeneous Poisson Process (NHPP) and manually calibrated the time and day multipliers to accurately reflect empirical banking traffic waves.

- **Simulation Engineering:** I developed the core Python logic, programming the customer lifecycle, the priority queue routing, and the automated Grid Search marginal analysis loop.
- **Data Analysis & Strategy:** I analyzed the raw simulation data to mathematically prove the failure points of the baseline system, ultimately translating the Grid Search outputs into the final business recommendation (7 Tellers, 3 Helpdesk).

5 References

1. **SimPy Documentation:** Discrete event simulation framework for Python. Available at: <https://simpy.readthedocs.io/>
2. **SciPy Documentation:** Statistical functions used for Gamma, Lognormal, and Weibull distributions. Available at: <https://docs.scipy.org/doc/scipy/reference/stats.html>
3. **Non-Homogeneous Poisson Processes (NHPP):** Mathematical theory for modeling time-dependent arrival rates. Available at: https://en.wikipedia.org/wiki/Poisson_point_process#Non-homogeneous_Poisson_process
4. **Queuing Theory & Balking/Reneging Concepts:** Fundamentals of operations research and human behavior in queues. Available at: https://en.wikipedia.org/wiki/Queueing_theory
5. Google Gemini

Deliverables

Code & Repository Access Link: <https://github.com/ak160/Bank-Simulation>